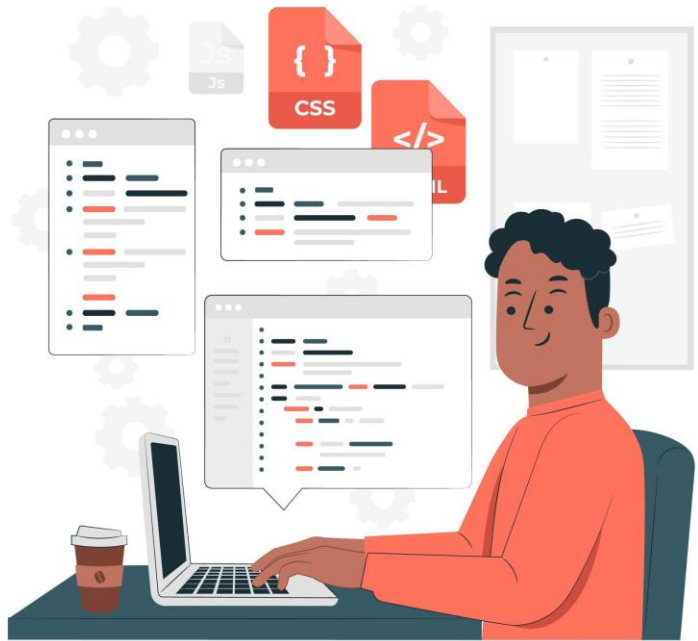

CORS & REST API dengan ORM Prisma





KEMNAKER

Outline

- Apa itu CORS
- CORS di express.js
- Prisma Schema
- Prisma Client
- Prisma Migrate
- REST API dengan prisma

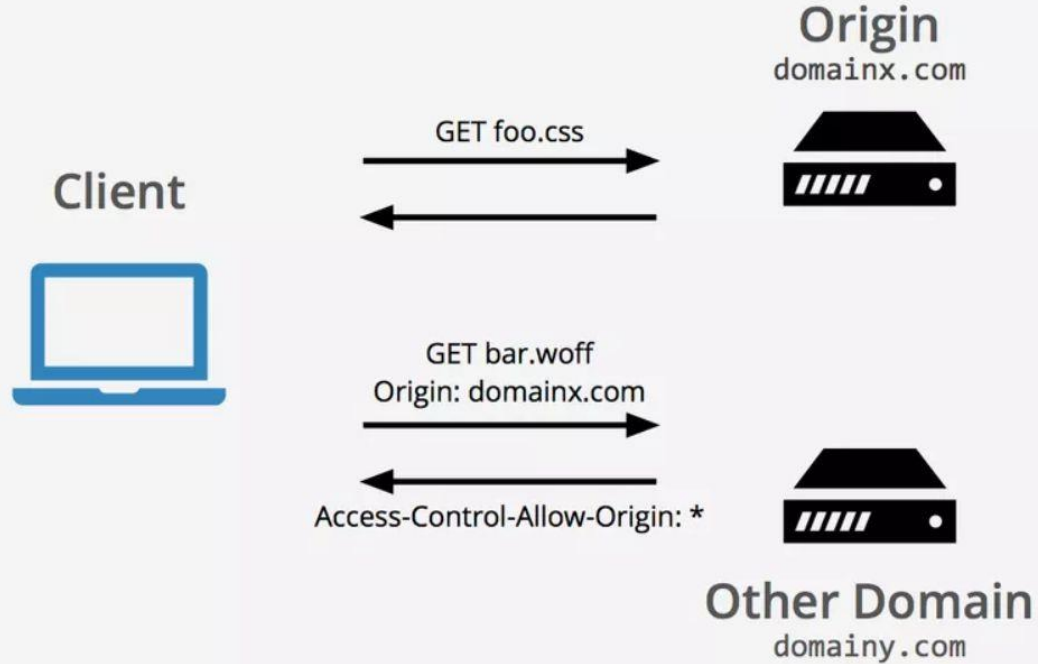
CORS

Apa itu CORS?

CORS di JavaScript adalah singkatan dari **Cross-Origin Resource Sharing**. Ini adalah **mekanisme keamanan di browser** yang mengatur apakah sebuah website boleh mengakses data/resource dari domain lain.

yang biasanya jika CORS aktif memiliki response HTTP di bagian header seperti : Access-Control-Allow-Origin: *

Namun, ini juga memberikan potensi serangan berbasis domain, jika kebijakan CORS situs web tidak dikonfigurasi dan diterapkan dengan baik. CORS bukan perlindungan terhadap serangan lintas-asal seperti pemalsuan permintaan lintas situs (CSRF).



Gambaran Sederhana

- Kalau kamu buka website `https://contoh.com` dan website ini mencoba mengambil data dari `https://api.lain.com`, maka itu disebut **cross-origin request** (karena domainnya beda).
- Secara default, browser **memblokir** permintaan semacam ini demi keamanan (supaya website berbahaya tidak bisa sembarangan ambil data dari server lain).
- Nah, **CORS** adalah aturan/protokol yang memungkinkan server `https://api.lain.com` memberi izin bahwa “boleh kok diakses oleh `https://contoh.com`”

Kapan pakai CORS ?

Perlu setting **CORS** di server (backend) jika FrontEnd dan BackEnd dipisahkan (dua server / microservice style)

Kalau frontend & backend beda port (walaupun 1 folder) → tetap butuh CORS

Contoh saat development:

React jalan di `http://localhost:3000`

Express jalan di `http://localhost:5000`

Ini dianggap **beda origin** (karena beda port), jadi browser butuh aturan CORS dari backend.

Dev mode (frontend & backend beda port) → butuh CORS.

Prod mode (gabung satu port/domain) → tidak butuh CORS.

Cara Kerja CORS

Browser (<https://contoh.com>)

- |
 - └ Request ke <https://api.lain.com/data>
 - | (dikirim dengan header **Origin**: <https://contoh.com>)
- |
 - └ Server api.lain.com harus jawab:
 - Access-Control-Allow-Origin**: <https://contoh.com>

Same-Origin:

[contoh.com] <—> [contoh.com]  OK

Cross-Origin:

[contoh.com] <—X—> [api.lain.com]  Blokir (kecuali server aktifkan CORS)

Cara Kerja CORS

1. Browser mengirim request ke server dengan menyertakan **Origin header**
→ Origin: `https://contoh.com`
2. Server memutuskan apakah origin tersebut diizinkan atau tidak.
3. Jika diizinkan, server mengirim response dengan header CORS, misalnya:

```
Access-Control-Allow-Origin: https://contoh.com
```

Atau

```
Access-Control-Allow-Origin: *
```

(* artinya semua origin boleh akses)

Kalau header itu ada dan valid → browser izinkan JavaScript mengambil datanya.
Kalau tidak → browser blokir dan muncul error CORS di console.

Implementasi CORS pada express.js

langkah pertama yang dilakukan adalah instalasi library cors dari express.js:
npm install cors

lalu implementasi seperti dibawah ini:

```
const cors = require('cors')

var corsOptions = {
  origin: 'http://localhost:3000',
  optionsSuccessStatus: 200
}

app.use(cors(corsOptions))
```

ORM Prisma

ORM (Object Relational Mapping)

Menghubungkan database relasional (MySQL, PostgreSQL, SQLite, SQL Server, dll.) dengan bahasa pemrograman (JavaScript/TypeScript) melalui objek.

***TypeScript** adalah **bahasa pemrograman** yang merupakan **superset dari JavaScript**.

Artinya: semua kode JavaScript valid juga di TypeScript, tapi TypeScript menambahkan **fitur baru** yang tidak ada di JavaScript.

TypeScript = JavaScript + fitur tambahan.

Apa itu Prisma?

<https://www.prisma.io/docs/concepts/overview/what-is-prisma>

<https://www.prisma.io/docs/getting-started>

- ORM **Prisma** adalah sebuah **Object-Relational Mapping (ORM)** modern untuk **Node.js** dan **TypeScript** yang mempermudah interaksi dengan database.
- Kalau biasanya kita menulis query SQL secara manual (SELECT * FROM users WHERE ...), dengan Prisma kita bisa menulis query menggunakan **JavaScript/TypeScript API** yang lebih aman, singkat, dan mudah dibaca.

Apa itu Prisma?

Prisma secara garis besar terdiri dari bagian-bagian berikut:

- **Prisma Schema (schema.prisma)** : Blueprint
- **Prisma Client**: Auto-generated and type-safe query builder for Node.js & TypeScript
- **Prisma Migrate**: Migration system
- **Prisma Studio**: GUI to view and edit data in your database.

Prisma Studio is the only part of Prisma ORM that is not open source.
You can only run Prisma Studio locally.

Prisma Schema (schema.prisma)

- Bagaimana prisma bekerja dimulai dari sini.
Prisma Schema file merupakan konfigurasi utama dalam prisma, di sana kita bisa membuat model data yang nantinya bisa kita gunakan.
- File utama untuk mendefinisikan model (tabel), relasi, dan konfigurasi database.
Dari sini Prisma akan menghasilkan query builder otomatis.
→ Mirip dengan **blueprint** untuk membuat tabel.

<https://www.prisma.io/docs/concepts/components/prisma-schema>

Contoh:

Prisma Schema (schema.prisma)

File utama untuk mendefinisikan model (tabel), relasi, dan konfigurasi database.

Dari sini Prisma akan menghasilkan query builder otomatis.

→ Mirip dengan **blueprint** untuk membuat tabel.

```
</> prisma

datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id    Int    @id @default(autoincrement())
  name  String
  email String  @unique
  posts Post[]
}

model Post {
  id      Int    @id @default(autoincrement())
  title   String
  content String?
  user    User   @relation(fields: [userId], references: [id])
  userId  Int
}
```

→ Mirip dengan **blueprint** untuk membuat tabel.

Prisma Client

Prisma Client adalah auto-generated dan type-safe query builder yang disesuaikan dengan model data yang sudah dibuat lewat prisma schema.

Prisma Client adalah **library JavaScript/TypeScript yang dihasilkan otomatis**. Secara otomatis Prisma akan generate sebuah library client berdasarkan schema.

Berisi kode untuk query database tanpa harus nulis SQL manual. Client ini digunakan untuk CRUD data tanpa perlu nulis SQL mentah.

<https://www.prisma.io/docs/concepts/components/prisma-client>

Prisma Migrate

- Migration System.
- Prisma Migrate adalah tahapan dimana kita migrasi dari apa yang sudah kita tuliskan di prisma schema ke database, bisa berupa membuat tabel baru, alter tabel dan lain sebagainya.
- Prisma bisa melakukan migrasi database otomatis (`npx prisma migrate dev`) untuk membuat atau update tabel sesuai schema.

<https://www.prisma.io/docs/concepts/components/prisma-migrate>

Kelebihan Prisma dibanding ORM lain (Sequelize, TypeORM)

- **Type-safe** → kalau pakai TypeScript, Prisma otomatis memberi autocomplete dan cek error di waktu compile.
- **Mudah dibaca** → query lebih mirip bahasa natural daripada raw SQL.
- **Modern & Cepat** → dukungan database populer, dokumentasi jelas, tool seperti Prisma Studio (GUI untuk lihat data).
- **Migration otomatis** → gampang sinkronisasi model dengan database.

WORKFLOW PRISMA

`npm install --save-dev prisma`

Instal CLI(Command Line Interface) untuk perintah `npx prisma ...` yang kamu pakai saat development

cth :

`npx prisma init`
`npx prisma migrate dev`
`npx prisma generate`
`npx prisma studio`

`npm install @prisma/client`

Instal di dependency agar saat app jalan dapat menggunakan query ke DB

WORKFLOW PRISMA

1. Init project :	<i>npx prisma init</i>	1. buat folder prisma/ dan file schema.prisma 2. membuat struktur awal (default setup) dari proyek Prisma — termasuk file schema.prisma yang nanti kamu ubah sesuai kebutuhan.
		 prisma/ └─ schema.prisma ← file utama untuk definisi model database  .env ← file untuk menyimpan koneksi database
2. Edit schema.prisma sesuai model database		
3. Buat dan jalankan migrasi:	<i>npx prisma migrate dev --name init</i>	Migrasi DB atau Update schema + DB + Prisma Client → otomatis juga jalankan generate (npx prisma generate) kalau dijalankan setelah merubah kode di schema prisma --> digunakan untuk mengubah struktur database (bukan hanya file di kode).
4. Update Prisma Client	<i>npx prisma generate</i>	Update Prisma Client saja Membangun atau memperbarui Prisma Client berdasarkan isi file schema.prisma. Agar Prisma membaca blueprint di schema.prisma dan membangkitkan (generate) library JS otomatis di: node_modules/@prisma/client/
5. Gunakan Prisma Client di kode:		
<pre>import { PrismaClient } from '@prisma/client'; const prisma = new PrismaClient();</pre>		

Demo membuat REST API dengan ORM Prisma

Struktur Folder Aplikasi

```
project-root/
├─ prisma/
│  └─ schema.prisma      <-- (setting data - pengganti pembuatan db)
├─ src/
│  ├─ config/
│  │  └─ utils.js        <-- Prisma client export
│  ├─ middleware/
│  │  └─ validation.js   <-- middleware validasi
│  ├─ controllers/
│  │  ├─ movieController.js
│  │  └─ categoryController.js
│  ├─ routes/
│  │  ├─ movieRoutes.js
│  │  └─ categoryRoutes.js
│  └─ index.js           <-- express app
├─ .env
├─ .gitignore
└─ package.json
```

package.json & dependency install

Inisialisasi dan buat file package.json dan starter pack aps

```
npm init -y
```

Install dependencies

```
npm install express cors @prisma/client
```

```
npm install --save-dev prisma nodemon
```

- prisma → tools/CLI buat migrate, generate, dll (hanya dipakai development).
- @prisma/client → library yang dipakai aplikasimu di runtime/production untuk query DB.

--save-dev artinya package yang kamu install **hanya dipakai saat development**, bukan saat aplikasi dijalankan di production. Jadi package itu masuk ke **devDependencies** di file package.json.

package.json & dependency install

Tambahkan pada scripts *optional:

```
"dev": "nodemon src/index.js",  
"start": "node src/index.js",  
"prisma:generate": "prisma generate",  
"prisma:migrate": "prisma migrate dev"
```

```
{  
  "name": "latexpressormprisma",  
  "version": "1.0.0",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \\\"Error: no test specified\\\" && exit 1",  
    "dev": "nodemon src/index.js",  
    "start": "node src/index.js",  
    "prisma:generate": "prisma generate",  
    "prisma:migrate": "prisma migrate dev"  
  },  
  "dependencies": {  
    "@prisma/client": "^6.14.0",  
    "cors": "^2.8.5",  
    "express": "^5.1.0"  
  },  
  "devDependencies": {  
    "nodemon": "^3.1.10",  
    "prisma": "^6.14.0"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "description": ""  
}
```

prisma/schema.prisma

Terminal :

npx prisma init

Perintah ini akan otomatis:

- Membuat folder prisma/
- Di dalamnya ada file prisma/schema.prisma
- Membuat file .env untuk menyimpan connection string database.
- Menambahkan dependensi Prisma (kalau belum)

Catatan: taruh DATABASE_URL di file .env (contoh nanti).

```
prisma > schema.prisma
1  generator client {
2    provider = "prisma-client-js"
3  }
4
5  datasource db {
6    provider = "mysql"
7    url      = env("DATABASE_URL")
8  }
9
10 model Category {
11   id        Int      @id @default(autoincrement())
12   name      String
13   description String?
14   createdAt DateTime @default(now())
15   updatedAt DateTime @updatedAt @default(now())
16   movies    Movie[]
17 }
18
19 model Movie {
20   id        Int      @id @default(autoincrement())
21   title     String
22   year      Int
23   createdAt DateTime @default(now())
24   updatedAt DateTime @updatedAt @default(now())
25   category  Category? @relation(fields: [categoryId], references: [id], onDelete: SetNull)
26   categoryId Int?
27 }
28
```

Prisma Memiliki Loader .env Sendiri jadi bisa langsung baca tanpa pakai instal dotenv

Setup Prisma & database (perintah)

- `schema.prisma` = blueprint
- `migrate dev` = tukang bangunan (bangun database dari blueprint)
- `generate` = buat remote control untuk database-nya di kode JavaScript

Setup Prisma & database (perintah)

1. Tambahkan .env (contoh):

```
DATABASE_URL="mysql://user:password@localhost:3306/dbname"
```

Contoh : DATABASE_URL="mysql://userroot:1234567@localhost:3306/db_movie_prisma"

Setup Prisma & database (perintah)

2. Jalankan migration & buat tabel:

```
npx prisma migrate dev --name init
```

Perintah di atas akan membuat folder prisma/migrations (history migrasi). Setiap ada perubahan bisa di migrate lagi dan nama bisa direname sesuai keterangan kita.

Database akan ada di computer kita.

npx prisma migrate dev

Ini perintah untuk **membuat dan menerapkan perubahan di database** sesuai isi schema.prisma.

Fungsinya:

- Membuat file migrasi SQL di folder prisma/migrations
- Menjalankan perintah SQL ke database (membuat tabel, ubah kolom, dll)
- **Setelah selesai, otomatis memanggil prisma generate** untuk memperbarui Prisma Client.

Jadi migrate dev = update database + generate client.

Setup Prisma & database (perintah)

3. Generate client

`npx prisma generate`

- Perintah ini akan membuat folder **node_modules/.prisma/client**
- Isinya adalah **kode client** yang dibutuhkan agar aplikasi bisa query database pakai Prisma.
- Prisma Client itu dibangkitkan dari definisi di `schema.prisma`.

*Klien ini dihasilkan secara otomatis dari definisi skema Prisma kita, melalui perintah seperti `prisma generate` -> memastikan klien selalu selaras dengan struktur basis data Anda.

npx prisma generate

Tujuan: Membuat *Prisma Client* berdasarkan `schema.prisma`.

Kalau dijalankan setelah ubah `schema` dan `migrate` → Ini cuma membuat ulang **Prisma Client** (kode akses database di `node_modules/@prisma/client`)

Tapi **tidak mengubah database** sama sekali.

Fungsi utamanya:

- Membaca semua model di `schema.prisma`
- Menghasilkan kode Prisma Client di folder `node_modules/@prisma/client`
- Supaya bisa akses database dengan kode seperti ini:
 - `import { PrismaClient } from '@prisma/client'`
 - `const prisma = new PrismaClient()`

Biasanya jalankan:

- Setelah mengubah **`schema.prisma`**
(misal tambah model baru, ubah field, dll)
- Setelah menjalankan `npx prisma migrate dev`
- Atau saat mau memastikan Prisma Client terbaru sudah digenerate.

*** Kenapa kadang setelah prisma init langsung muncul folder client?**

Itu bisa terjadi karena beberapa alasan:

1. Kamu **sudah pernah install @prisma/client** sebelumnya dengan `npm install @prisma/client`
2. Saat `@prisma/client` diinstal, **`npm postinstall script`** otomatis menjalankan: `prisma generate`

src/config/utils.js — konfigurasi Prisma client

```
src > config > JS utils.js > ...
1  // src/config/utils.js
2
3  const { PrismaClient } = require('@prisma/client');
4  //Import class PrismaClient dari package @prisma/client (generator pada prisma/schema.prisma membuat package ini).
5
6  const prisma = new PrismaClient();
7  //Inisialisasi instance Prisma yang terhubung ke DB sesuai DATABASE_URL.
8
9  module.exports = prisma;
10 //Ekspor instance agar file lain (controllers) bisa require('../config/utils') dan memakai prisma.
11
```

src/middleware/validation.js — middleware validasi

```
src > middleware > JS validation.js > ...
1  // src/middleware/validation.js
2  const validationBodyMovies = (req, res, next) => {
3    let { title, year } = req.body;
4
5    if (title === undefined || year === undefined) {
6      res.status(400).json({ message: "title and year is required" });
7    } else {
8      next();
9    }
10 }
11
12 const validationBodyCategories = (req, res, next) => {
13   let { name } = req.body;
14
15   if (name === undefined) {
16     res.status(400).json({ message: "name is required" });
17   } else {
18     next();
19   }
20 }
21
22 module.exports = {
23   ...validationBodyMovies,
24   validationBodyCategories
25 }
26 //Ekspor middleware untuk dipakai di route.
27
```

src/controllers/movieController.js — controller CRUD Movie (async)

```
src > controllers > JS movieController.js > ...  
1  // src/controllers/movieController.js  
2  const prisma = require('../config/utils');  
3
```

src/controllers/movieController.js — controller CRUD Movie (async)

```
4  //READ
5  const getAllMovies = async (req, res) => {
6    try {
7      const movies = await prisma.movie.findMany({
8        include: { category: true } // sertakan data category
9      });
10     return res.json(movies);
11   } catch (error) {
12     console.error(error);
13     return res.status(500).json({ message: 'Internal server error' });
14   }
15 }
16
17 const getMovieById = async (req, res) => {
18   try {
19     const id = parseInt(req.params.id);
20     const movie = await prisma.movie.findUnique({
21       where: { id },
22       include: { category: true }
23     });
24
25     if (!movie) return res.status(404).json({ message: 'Movie not found' });
26     return res.json(movie);
27   } catch (error) {
28     console.error(error);
29     return res.status(500).json({ message: 'Internal server error' });
30   }
31 }
```

src/controllers/movieController.js — controller CRUD Movie (async)

```
src > controllers > JS movieController.js > ...
33 //CREATE
34 const createMovie = async (req, res) => {
35   try {
36     const { title, year, categoryId } = req.body;
37     const data = { title, year: parseInt(year) };
38
39     // jika categoryId disertakan, set categoryId
40     if (categoryId !== undefined && categoryId !== null) {
41       data.categoryId = parseInt(categoryId);
42     }
43
44     const movie = await prisma.movie.create({
45       data,
46       include: { category: true }
47     });
48
49     return res.status(201).json(movie);
50   } catch (error) {
51     console.error(error);
52     // jika foreign key invalid, prisma akan error -> kirim 400
53     return res.status(400).json({ message: error.message });
54   }
55 }
```

src/controllers/movieController.js — controller CRUD Movie (async)

```
JS movieController.js X
LatExpressORMPrisma > src > controllers > JS movieController.js > ...
56
57 //UPDATE
58 const updateMovie = async (req, res) => {
59   try {
60     const id = parseInt(req.params.id);
61     const { title, year, categoryId } = req.body;
62     // siapkan data update; jika categoryId tidak dikirim, jangan ubah nilai categoryId
63     const data = { title, year: parseInt(year) };
64     if ('categoryId' in req.body) {
65       // in akan true kalau key ada di object (baik nilainya null, undefined, atau angka).
66       data.categoryId = categoryId === null ? null : parseInt(categoryId);
67       //ternary operator
68       // Kalau categoryId dikirim null → simpan null.
69       // Kalau categoryId dikirim angka → parseInt biar jadi number.
70     }
71
72     const movie = await prisma.movie.update({
73       where: { id },
74       data,
75       include: { category: true }
76     });
77
78     return res.json(movie);
79   } catch (error) {
80     console.error(error);
81     if (error.code === 'P2025') { //P2025 error code dari prisma kalau record not found
82       return res.status(404).json({ message: 'Movie not found' });
83     }
84     return res.status(400).json({ message: error.message });
85   }
86 }
87
```

P2025 adalah error code bawaan Prisma.
Artinya: "Record not found" (record tidak ada di database sesuai query where).

src/controllers/movieController.js — controller CRUD Movie (async)

```
src > controllers > JS movieController.js > ...
85
86 //DELETE
87 const deleteMovie = async (req, res) => {
88   try {
89     const id = parseInt(req.params.id);
90     await prisma.movie.delete({ where: { id } });
91     return res.json({ message: 'Movie deleted' });
92   } catch (error) {
93     console.error(error);
94     if (error.code === 'P2025') {
95       return res.status(404).json({ message: 'Movie not found' });
96     }
97     return res.status(500).json({ message: 'Internal server error' });
98   }
99 }
100
```

P2025 adalah error code bawaan Prisma.

Artinya: "Record not found" (record tidak ada di database sesuai query where).

src/controllers/movieController.js — controller CRUD Movie (async)

```
src > controllers > JS movieController.js >  
101   module.exports = {  
102     getAllMovies,  
103     getMovieById,  
104     createMovie,  
105     updateMovie,  
106     deleteMovie  
107   };  
108
```

src/controllers/categoryController.js — controller CRUD Category

```
src > controllers > JS categoryController.js > ...  
1  // src/controllers/categoryController.js  
2  const prisma = require('...../config/utils');  
3
```

src/controllers/categoryController.js — controller CRUD Category

```
src > controllers > JS categoryController.js > ...
```

```
4  //READ
5  const getAllCategories = async (req, res) => {
6    try {
7      const categories = await prisma.category.findMany({
8        include: { movies: true } // sertakan movies yang berhubungan
9      });
10     return res.json(categories);
11   } catch (error) {
12     console.error(error);
13     return res.status(500).json({ message: 'Internal server error' });
14   }
15 }
```

src/controllers/categoryController.js — controller CRUD Category

src > controllers > JS categoryController.js > ...

```
17  const getCategoryById = async (req, res) => {
18    try {
19      const id = parseInt(req.params.id);
20      const category = await prisma.category.findUnique({
21        where: { id },
22        include: { movies: true }
23      });
24
25      if (!category) return res.status(404).json({ message: 'Category not found' });
26      return res.json(category);
27    } catch (error) {
28      console.error(error);
29      return res.status(500).json({ message: 'Internal server error' });
30    }
31  }
```

src/controllers/categoryController.js — controller CRUD Category

```
src > controllers > JS categoryController.js > ...
```

```
33  //CREATE
34  const createCategory = async (req, res) => {
35    try {
36      const { name, description } = req.body;
37      const category = await prisma.category.create({
38        data: { name, description }
39      });
40      return res.status(201).json(category);
41    } catch (error) {
42      console.error(error);
43      return res.status(400).json({ message: error.message });
44    }
45  }
46
```

src/controllers/categoryController.js — controller CRUD Category

```
src > controllers > JS categoryController.js > ...
46 |
47 | //UPDATE
48 | const updateCategory = async (req, res) => {
49 |   try {
50 |     const id = parseInt(req.params.id);
51 |     const { name, description } = req.body;
52 |
53 |     const category = await prisma.category.update({
54 |       where: { id },
55 |       data: { name, description }
56 |     });
57 |     return res.json(category);
58 |   } catch (error) {
59 |     console.error(error);
60 |     if (error.code === 'P2025') {
61 |       return res.status(404).json({ message: 'Category not found' });
62 |     }
63 |     return res.status(400).json({ message: error.message });
64 |   }
65 | }
66 |
```

src/controllers/categoryController.js — controller CRUD Category

src > controllers > JS categoryController.js > ...

```
67 //DELETE
68 const deleteCategory = async (req, res) => {
69   try {
70     const id = parseInt(req.params.id);
71     // Karena relation di prisma onDelete: SetNull, saat delete category, movie.categoryId akan jadi null
72     await prisma.category.delete({ where: { id } });
73     return res.json({ message: 'Category deleted' });
74   } catch (error) {
75     console.error(error);
76     if (error.code === 'P2025') {
77       return res.status(404).json({ message: 'Category not found' });
78     }
79     return res.status(500).json({ message: 'Internal server error' });
80   }
81 }
```

src/controllers/categoryController.js — controller CRUD Category

```
src > controllers > JS categoryController.js > ...
```

```
83  module.exports = {  
84    getAllCategories,  
85    getCategoryById,  
86    createCategory,  
87    updateCategory,  
88    deleteCategory  
89  };
```


src/routes/movieRoutes.js

```
src > routes > JS movieRoutes.js > ...
1  // src/routes/movieRoutes.js
2  const express = require('express');
3  const router = express.Router();
4  const movieController = require('../controllers/movieController');
5  const { validationBodyMovies } = require('../middleware/validation');
6
7  router.get('/', movieController.getAllMovies);
8  router.get('/:id', movieController.getMovieById);
9  router.post('/', validationBodyMovies, movieController.createMovie);
10 router.put('/:id', validationBodyMovies, movieController.updateMovie);
11 router.delete('/:id', movieController.deleteMovie);
12
13 module.exports = router;
14
```

src/routes/categoryRoutes.js

```
src > routes > JS categoryRoutes.js > ...
```

```
1  // src/routes/categoryRoutes.js
2  const express = require('express');
3  const router = express.Router();
4  const categoryController = require('../controllers/categoryController');
5  const { validationBodyCategories } = require('../middleware/validation');
6
7  router.get('/', categoryController.getAllCategories);
8  router.get('/:id', categoryController.getCategoryById);
9  router.post('/', validationBodyCategories, categoryController.createCategory);
10 router.put('/:id', validationBodyCategories, categoryController.updateCategory);
11 router.delete('/:id', categoryController.deleteCategory);
12
13 module.exports = router;
```

src/routes/movieRoutes.js & src/routes/categoryRoutes.js

Penjelasan singkat per file

- Import express-router, controller, dan middleware.
- Pasang middleware validasi hanya pada endpoint yang membutuhkan body (POST, PUT).
- Ekspor router untuk dipakai di index.js.

src/index.js

— Express app utama

```
src > JS index.js > ...
1 // src/index.js
2 const express = require('express');
3 const cors = require('cors');
4 // const express = require('express'); → import Express.
5 // const cors = require('cors'); → import CORS package.
6
7 const app = express();
8
9 const movieRoutes = require('./routes/movieRoutes');
10 const categoryRoutes = require('./routes/categoryRoutes');
11
12 // Middlewares global
13 app.use(cors()); // enable CORS untuk semua origin
14 // app.use(cors()); → aktifkan CORS (default: semua origin diperbolehkan).
15 // Berguna saat frontend di origin berbeda.
16
17 app.use(express.json()); // parse JSON body
18
19 // Prefix API
20 app.use('/api/movies', movieRoutes);
21 app.use('/api/categories', categoryRoutes);
22 // app.use('/api/movies', movieRoutes); → mount route movie di prefix /api/movies.
23 // app.use('/api/categories', categoryRoutes); → mount route category di /api/categories.
24
25 // health-check
26 app.get('/', (req, res) => {
27 |   res.send('API berjalan – gunakan /api/movies dan /api/categories');
28 | });
29
30 const PORT = process.env.PORT || 3000;
31 app.listen(PORT, () => {
32 |   console.log(`Server berjalan pada port ${PORT}`);
33 | });
34
```

Cek Database

1. Cmd by administrator

- Cd C:\Program Files\MySQL\MySQL Server 8.0\bin
- mysql -u root -p
- Masukkan password
- Show databases;
- Use dbName;
- Select * from tableName;

2. Melalui Prisma Studio

Terminal vsCode jalankan : **npx prisma studio**

.gitignore

node_modules

.env

prisma/migrations

Jalankan Server

Jalankan server:

`npm run dev`

`# atau`

`npm start`

Updating data - Prisma Migrate

edit schema → jalankan prisma migrate dev → Prisma generate migrasi SQL → Prisma Client terupdate.

1. Edit data pada schema.prisma
2. Lalu di terminal : `npx prisma migrate dev --name add-keterangan-migrate`

Terminal :

```
npx prisma migrate dev --name <nama_migrasi>
```


Updating data - Prisma Migrate

1. Membandingkan schema lama dan baru
2. Membuat file SQL migration di folder prisma/migrations/
3. Menjalankan SQL-nya ke database, misalnya:

```
ALTER TABLE User ADD COLUMN age INTEGER;
```
4. Setelah itu, **otomatis menjalankan prisma generate** untuk memperbarui Prisma Client sesuai schema baru.

Updating data - npx prisma generate

Tujuan: Membuat ulang *Prisma Client* berdasarkan `schema.prisma`.

Ini cuma membuat ulang **Prisma Client** (kode akses database di `node_modules/@prisma/client`)
Tapi **tidak mengubah database** sama sekali.

Biasanya dijalankan:

- Setelah mengubah **schema.prisma**
(misal nambah model baru, ubah field, dll)
- Setelah menjalankan `npx prisma migrate dev`
- Atau saat mau memastikan Prisma Client terbaru sudah digenerate.

Mana yang duluan ?

`migrate dev → generate` ✓ Disarankan

`migrate dev` otomatis `generate` juga

- Jalankan **`npx prisma migrate dev`** dulu, karena dia otomatis meng-*generate* client di akhir.
- Jalankan `prisma generate` **hanya kalau kamu tidak melakukan migrasi**, tapi ingin update client-nya.

Testing

Ikuti langkah ini di VS Code + Thunder Client (ext):

A. Setup environment / base URL

- Di Thunder Client → Environments → tambah environment `Local` :
 - `BASE_URL = http://localhost:3000/api`
- Gunakan environment ini ketika mengetes.

Testing

B. Urutan testing (collection)

1. GET all categories

- Method: GET
- URL: `{{BASE_URL}}/categories`
- Headers: tidak perlu (default).
- Expected: 200 OK, array (awal kosong `[]`).

2. POST create category

- Method: POST
- URL: `{{BASE_URL}}/categories`
- Headers: Content-Type: `application/json`
- Body (raw JSON):

json

 Copy code

```
{
  "name": "Sci-Fi",
  "description": "Science fiction movies"
}
```

- Expected: 201 Created, JSON object dengan id, name, description, createdAt, updatedAt.

3. GET all categories → sekarang akan ada 1 category dan `movies: []`.

Testing

4. GET all movies (awal)

- GET `{{BASE_URL}}/movies` → `[]`.

5. POST create movie (tanpa category)

- POST `{{BASE_URL}}/movies`
- Body:

json

 Copy code

```
{
  "title": "Generic Movie",
  "year": 2020
}
```

- Karena middleware `validationBodyMovies`, `title` dan `year` wajib — jika salah satunya hilang → `400`.

6. POST create movie (dengan category)

- POST `{{BASE_URL}}/movies`
- Body:

json

 Copy code

```
{
  "title": "Inception",
  "year": 2010,
  "categoryId": 1
}
```

- Expected: `201` dan response termasuk `category` object (karena `include: { category: true }`).

7. GET all movies → cek movie sekarang muncul dengan `category`.

Testing

8. GET movie by id

- GET `{{BASE_URL}}/movies/1` → 200 + object.

9. PUT update movie

- PUT `{{BASE_URL}}/movies/1`
- Body:

json

 Copy code

```
{
  "title": "Inception (Updated)",
  "year": 2011,
  "categoryId": null
}
```

- `categoryId: null` akan menghapus hubungan (set NULL). Response 200 dengan updated object.

10. DELETE movie

- DELETE `{{BASE_URL}}/movies/1` → 200 { message: 'Movie deleted' }.

11. DELETE category

- DELETE `{{BASE_URL}}/categories/1` → 200 { message: 'Category deleted' }.

Karena `onDelete: SetNull`, jika ada movie yg berhubungan, `categoryId` pada movie akan jadi `NULL`. Uji: buat movie dengan category, lalu hapus category dan cek movie tetap ada tapi `categoryId = null`.

Catatan error yang perlu dicek di Thunder Client

- POST /movies tanpa `title` atau `year` → 400 dengan message: "title and year is required".
- POST /categories tanpa `name` → 400.
- Coba get/put/delete dengan ID yang tidak ada → 404.



KEMNAKER



Kesimpulan

- Apa itu CORS
- CORS di express.js
- Prisma Schema
- Prisma Client
- Prisma Migrate
- REST API dengan prisma

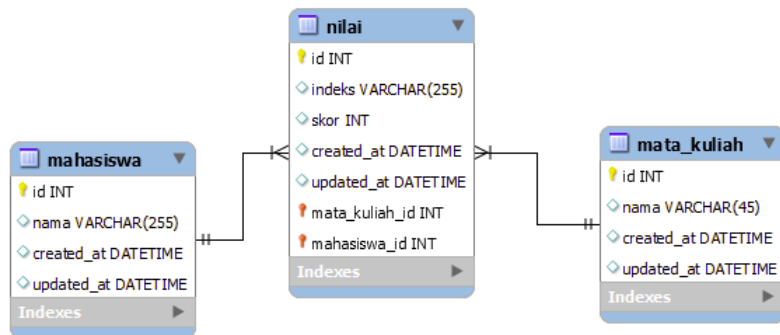
1. Menambahkan Folder baru dan file baru

Gunakanlah repository tugas (jangan buat repository baru lagi). Lalu buatlah 1 folder baru dengan nama tambahkan folder baru dengan nama **"tugas-rest-api-orm-prisma"**

2. Kerjakan Soal di bawah ini

kerjakan soal di bawah ini didalam folder **tugas-rest-api-orm-prisma**

buatlah CRUD REST API berdasarkan skema database dibawah ini



TUGAS

buat 3 CRUD berbeda sesuai dengan nama tabel masing-masing, untuk POST tabel nilai yang diperbolehkan di input dalam **body json** hanya **mata_kuliah_id**, **mahasiswa_id** dan **nilai**

lalu **nilai** hanya bisa diinput dari 0 s/d 100, jika diisi dibawah 0 atau diatas 100 maka akan ada pesan data nilai salah

untuk indeks didapatkan dari function yang ada dalam kode backend dengan ketentuan

nilai >= 80 indeksnya A,
nilai >= 70 dan nilai < 80 indeksnya B,
nilai >= 60 dan nilai < 70 indeksnya C,
nilai >= 50 dan nilai < 60 indeksnya D,
nilai < 50 indeksnya E

tugas diwajibkan menggunakan ORM Prisma

Terima Kasih

